

Agent Skills

COT4930/COT5930 Security Engineering with GenAI

What an agent skill contains

A reusable package of task instructions and supporting files

SKILL.md

Metadata identifies the skill and when it applies.

The Markdown body describes the procedure.

Host responsibilities

The agent harness discovers the package, loads its instructions, and provides execution tools.

Optional supporting files

Scripts implement repeatable operations. References and assets supply task material.

Scope of the mechanism

Loading a skill changes runtime context. It does not update the model's weights.

Discovery, activation, execution

Progressive disclosure limits how much material enters context



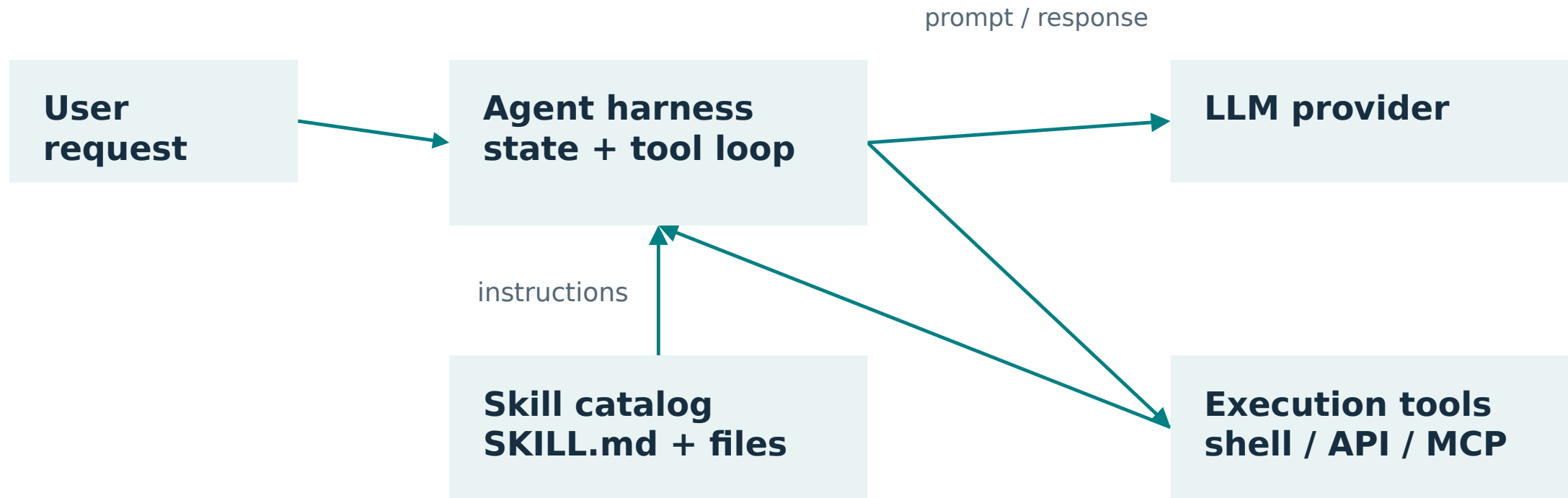
Task relevance

The model or host selects a skill.
The exact trigger policy depends on the application.

Skill instructions guide behavior. The harness determines which actions are actually possible.

The agent harness around a skill

Control flow and execution remain responsibilities of the application



A skill can use tools only when the host makes those tools available.

Skills and MCP address different concerns

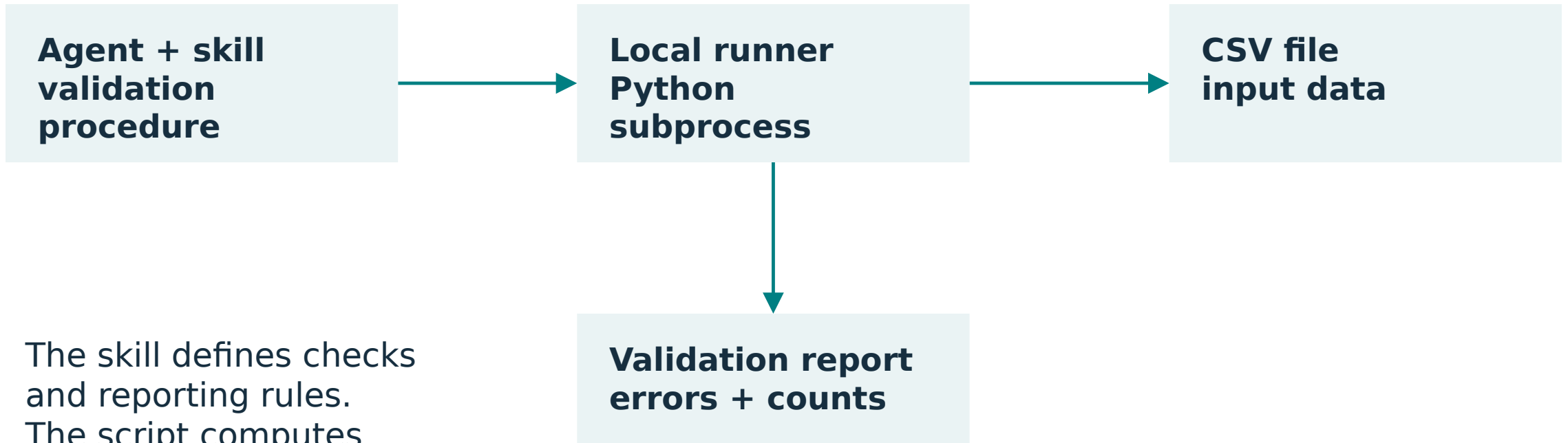
They can work independently or in combination

Aspect	Agent skill	MCP
Primary role	Task procedure and reusable context	Interface between a host and a server
Typical content	Instructions, scripts, references	Tool schemas, resources, prompts
Execution	Uses host-provided capabilities	Server handles protocol requests
Dependency	Can run with local tools alone	Can serve tools without any skills
Combination	Explains how to use an MCP tool	Supplies the tool the skill uses

MCP also carries prompts and resources. “Skills versus tools” is not the whole comparison.

Architecture A: local skill, no MCP

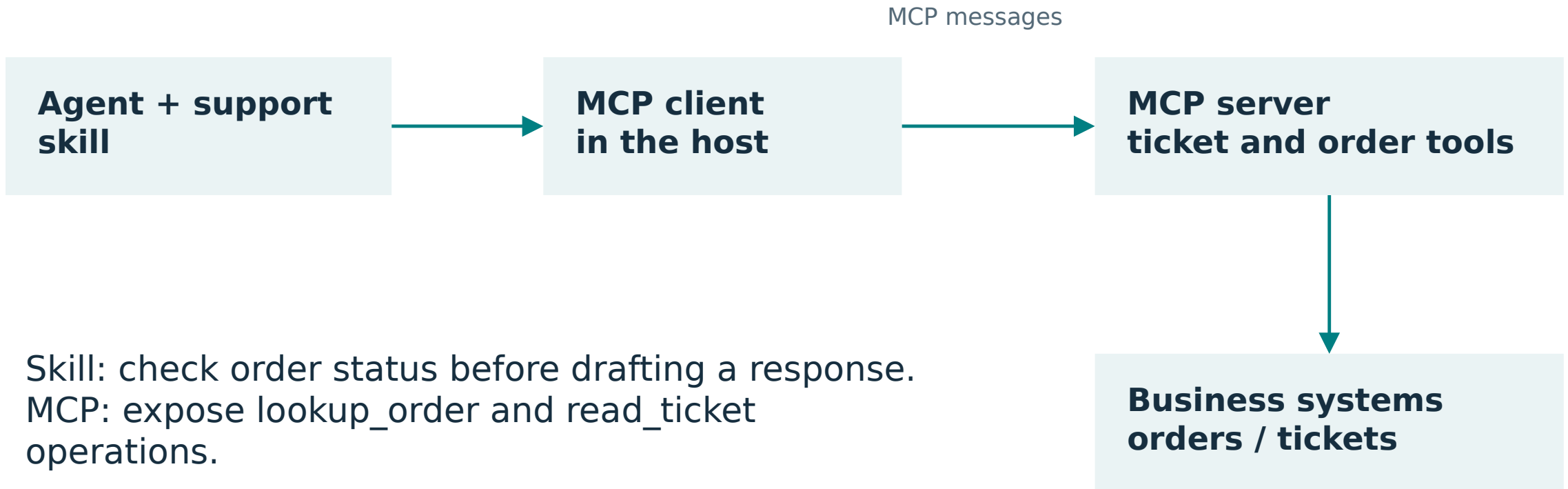
Example: inspecting a CSV with a bundled Python validator



The skill defines checks and reporting rules. The script computes results from the file.

Architecture B: skill with MCP

Example: preparing a customer-support answer



Access credentials and authorization belong to the execution system, not to the prose alone.

Practical choices

The same agent can combine several approaches

Task	Skill procedure	Execution mechanism
Summarize supplied text	Follow a style and evidence rubric	Model + provided text
Check a data file	Run validator, explain failures	Local Python script
Answer from PSU CS	Fetch page, inspect evidence, cite path	Python + curl + HTTPS
Prepare a support reply	Read ticket, check order, draft answer	MCP tools
Review a repository	Inspect diff, run tests, report findings	Local Git or a Git MCP server

Choose MCP when a standard server interface is useful. Skill packaging does not require it.

The pdx-cs-curl skill from Lab04 MCP & Skills

A source-restricted retrieval procedure supplied in the [Lab04 Codelabs page](#)

Trigger

Questions about Portland State University CS when evidence should come from web.cs.pdx.edu.

Action

Run the bundled script with an optional page path. Follow relevant links by fetching another page.

Evidence rule

Use only retrieved PSU CS content. If it lacks the answer, report that limitation.

Response contract

Provide the fact or summary, the page path used, and any caveats about the evidence.

Package layout and metadata

The host discovers SKILL.md; the script supplies the retrieval operation

```
skills/  
  pdx-cs-curl/  
    SKILL.md  
  scripts/  
    pdx_cs_curl.py
```

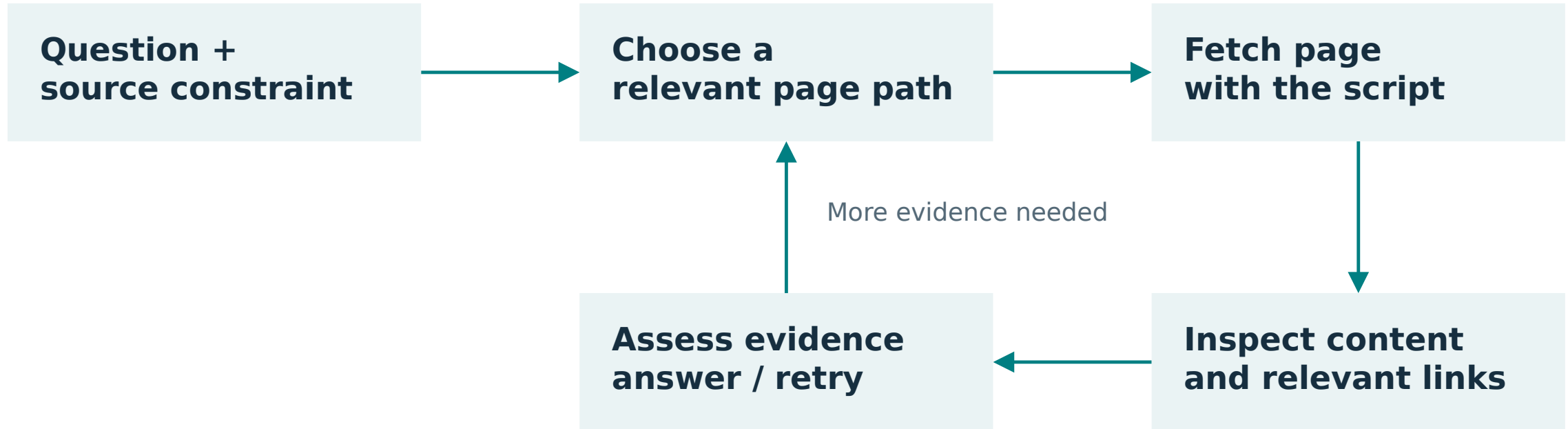
```
---  
name: pdx-cs-curl  
description: Use to answer questions  
             about Portland State University  
             CS department from web.cs.pdx.edu  
---
```

Run the client from the directory containing skills/.
The example command assumes this working directory.

The script is ordinary Python. SKILL.md tells the agent when and how to use it.

The skill's retrieval workflow

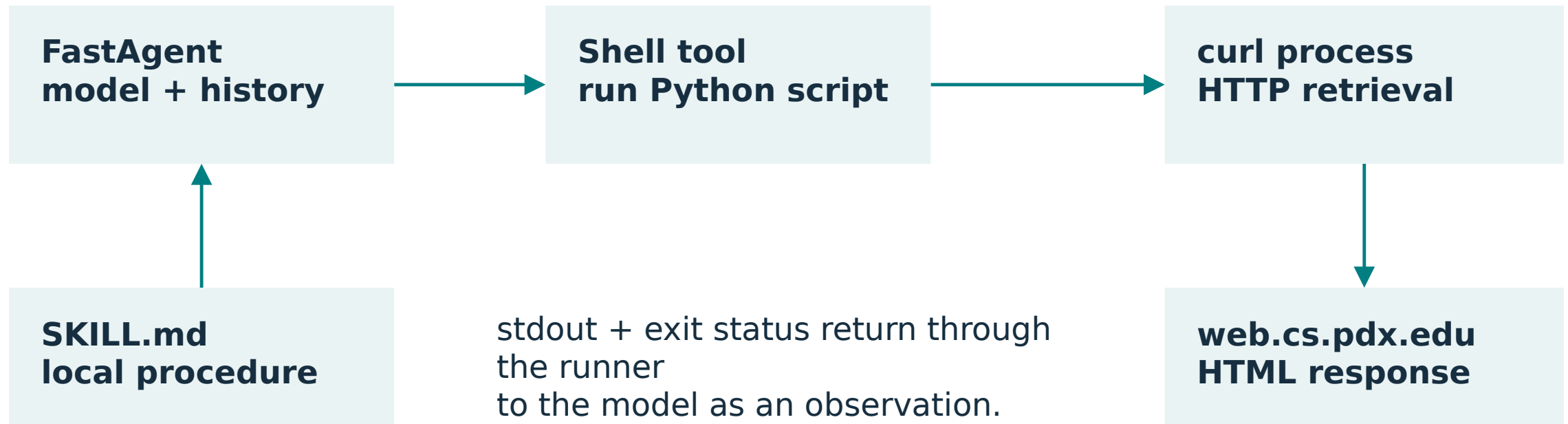
Paths must come from the task or discovered links, then be checked



A successful HTTP request does not prove that the page answers the question.

PDX architecture without an MCP server

The local script accesses the site directly over HTTPS



FastAgent provides the execution capability. The skill itself does not open a network connection.

Script entry point and arguments

main() returns a process exit status

```
BASE_URL = "https://web.cs.pdx.edu/"

def main() -> int:
    if len(sys.argv) > 2:
        print("usage: pdx_cs_curl.py [relative-path]",
              file=sys.stderr)
        return 2

    relative_path = (
        sys.argv[1] if len(sys.argv) == 2 else ""
    )

    if __name__ == "__main__":
        raise SystemExit(main())
```

sys.argv[0]
Script name

Zero or one argument
Empty path selects the
homepage

Exit status 2
Invalid command usage

Input validation and URL construction

The script rejects explicit external URLs and literal parent segments

```
parsed = urlparse(relative_path)
if parsed.scheme or parsed.netloc:
    print("relative-path must not be an absolute URL",
          file=sys.stderr)
    return 2

relative_path = parsed.path.lstrip("/")
if any(part == ".." for part in relative_path.split("/")):
    print("relative-path must not contain '..' segments",
          file=sys.stderr)
    return 2

url = urljoin(BASE_URL, relative_path)
```

scheme / netloc
Reject absolute URLs
and //host paths

lstrip("/")
Treat paths as relative

urljoin
Build the initial request
URL

Only parsed.path survives. Query strings, fragments, and path parameters are discarded.

The curl subprocess

A list of arguments avoids shell interpretation inside subprocess.run

```
result = subprocess.run(
    ["curl", "--silent", "--show-error",
     "--fail", "--location", url],
    capture_output=True,
    text=True,
)
```

Option	Effect
--silent + --show-error	Hide progress, retain failure diagnostics
--fail	Return an error for HTTP failure status codes
--location	Follow HTTP redirects
capture_output=True / text=True	Capture stdout and stderr as decoded strings

Results and failures

The caller receives page content or an error status

```
if result.returncode != 0:
    message = (result.stderr.strip() or
               f"curl failed with exit code {result.returncode}")
    print(message, file=sys.stderr)
    return result.returncode

print(result.stdout, end="")
return 0
```

Success
HTML body on
stdout
Exit status 0

Failure
Diagnostic on stderr
Nonzero exit status

Missing curl raises an exception. The supplied script has no timeout or output-size limit.

What the script guarantees and leaves open

The policy says “PSU CS only”; implementation needs stronger enforcement

Implemented checks

Reject a scheme or network location in the input.

Reject literal “..” path segments.

Pass curl arguments without shell=True.

Content limitations

Returns raw HTML, including navigation.

No JavaScript rendering or link extraction.

No automatic check of factual support.

Remaining gaps

Redirects can leave the allowed host.

Encoded paths need an explicit policy.

No total timeout or response-size bound.

Production improvements

Validate each redirect before following it.

Bound time and response size.

Return source URL and extracted text.

A practical PDX question

Illustrative execution trace

“Which faculty work on topic X? Cite the department page.”

```
python skills/pdx-cs-curl/scripts/pdx_cs_curl.py  
python skills/pdx-cs-curl/scripts/pdx_cs_curl.py faculty/
```

1. Inspect the homepage and locate the relevant link.
2. Fetch the linked page and extract statements about topic X.
3. Fetch a profile page if the directory lacks detail.
4. Report supported names and paths, or state that evidence is missing.

Do not infer a research area solely from a person's name or the model's prior knowledge.

FastAgent client startup

The client selects a model and registers the local skill directory

```
fast = FastAgent(  
    "PDX CS Curl Agent",  
    skills_directory="skills",  
)  
  
choices = []  
for model in ModelFactory.MODEL_PRESETS:  
    if ModelFactory.MODEL_PRESETS[model] == Provider.FAST_AGENT:  
        continue  
    choices.append(model)  
choices.append("custom")
```

skills_directory
Local discovery
location

MODEL_PRESETS
Choices from this
library version

Provider filter
Skips FAST_AGENT
entries

The preset list describes library configuration. It does not prove provider access or model availability.

Model selection and user input

questionary runs before the async agent loop

```
model = questionnaire.select(  
    "Select the model to use for the agent:",  
    choices=choices,  
).ask()  
  
if model == "custom":  
    model = questionnaire.text(  
        "Enter the model name ..."  
    ).ask()
```

The selected string
chooses
the model/provider route.

The matching API
credentials
must exist in the
environment
or provider configuration.

Validate cancellation and
empty custom input.

A robust client checks “if not model” before constructing the agent.

Prompt templates connect discovery to the model

FastAgent expands these placeholders before the model call

```
instruction="""You are a Portland State University  
computer science web assistant.  
Use the local pdx-cs-curl skill to fetch content  
from https://web.cs.pdx.edu.
```

```
{{agentSkills}}  
{{env}}
```

```
The current date is {{currentDate}}.  
"""
```

{{agentSkills}}
Skill metadata and
locations

{{env}}
Host/workspace context

{{currentDate}}
Date supplied by the host

These are framework template variables in a normal string, not Python f-string expressions.

The async client lifecycle

The decorator registers the agent; main() opens the interactive session

```
@fast.agent(  
    instruction=...,  
    model=model,  
    use_history=True,  
)  
async def main():  
    async with fast.run() as agent:  
        await agent.interactive()  
  
if __name__ == "__main__":  
    asyncio.run(main())
```

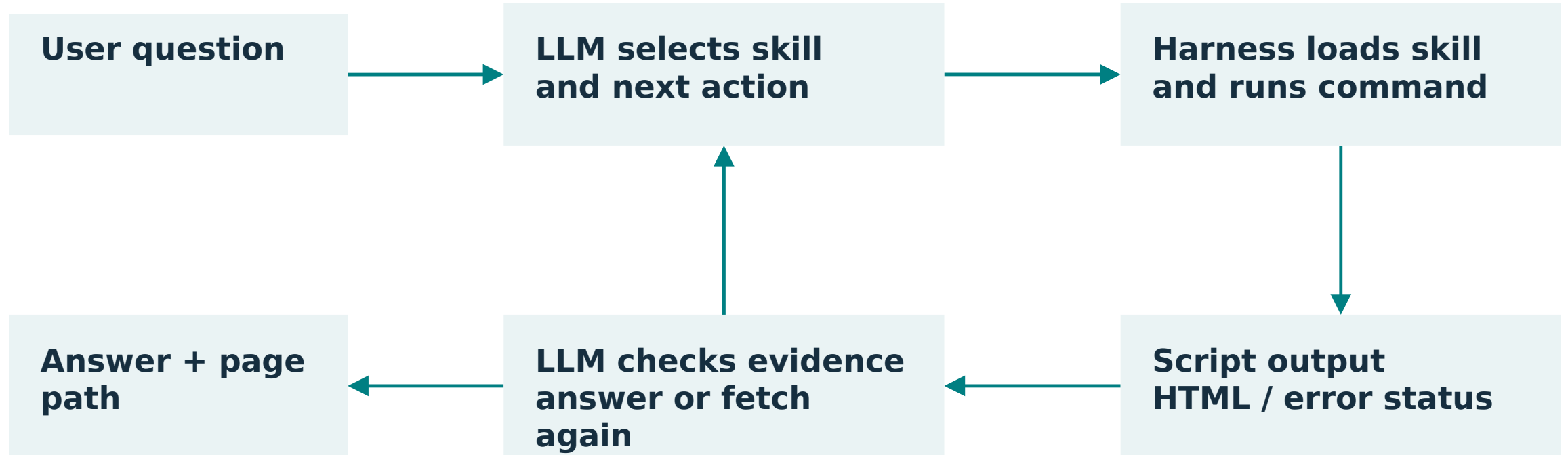
Register
Instructions + model +
history

Start runtime
Acquire agent resources

Interact and clean up
Await the session, then exit

One question through the client

Skill selection and tool execution happen inside the agent loop



The LLM chooses actions. The harness executes them. The script retrieves bytes.

Runtime requirements and a useful demonstration

A working skill needs an execution environment, not only a Markdown file

Required locally

Python, curl, FastAgent and questionnaire.
The skill files under skills/pdx-cs-curl/.
Outbound HTTPS and a provider API key.

Version-sensitive pieces

FastAgent imports and MODEL_PRESETS.
Skill discovery and shell activation.
Model aliases and provider credentials.

```
curl --version  
python skills/pdx-cs-curl/scripts/pdx_cs_curl.py  
uv run pdx_cs_client.py
```

First verify retrieval without an LLM, then verify discovery, execution, and evidence-based answers.

Evaluation cases for the supplied skill

Test input handling separately from the agent's answer quality

Case	Expected behavior
No path argument	Request the site homepage
https://example.org/ or //example.org/	Reject with exit status 2
../faculty/	Reject literal parent segment
faculty/?q=ai#people	Fetch faculty/ only, query and fragment discarded
Page missing or network failure	Report the fetch failure, do not invent facts
Page lacks the requested fact	State that the retrieved content is insufficient
Page contains instructions to the agent	Treat page text as evidence, not trusted instructions

An MCP variant of the same PDX workflow

A design alternative when several clients need a shared fetch service



This alternative adds a protocol service. The original local curl skill does not need one.

Design conclusions

Skills provide reusable procedures within an agent architecture

Procedure and capability

A skill can explain how to act.
The harness must supply the tools,
permissions, and runtime that make the
actions possible.

Evidence and enforcement

The PDX skill states a source policy.
Validate redirects and output handling if
that policy must hold mechanically.

MCP is an optional connection

Use local scripts for local operations.
Use MCP when standardized tool or data
access improves the application.

Evaluation

Check both script behavior and the agent's
answers. A correct fetch can still produce
an unsupported summary.

References and code provenance

Official documentation and the user-supplied example

Agent Skills overview and specification

<https://agentskills.io/home>

<https://agentskills.io/specification>

FastAgent skills and prompt templates

<https://fast-agent.ai/guides/skills/>

<https://fast-agent.ai/agents/instructions/>

MCP server primitives

<https://modelcontextprotocol.io/specification/draft/server/index>

Python subprocess and URL handling

<https://docs.python.org/3/library/subprocess.html>

<https://docs.python.org/3/library/urllib.parse.html>

curl option reference

<https://curl.se/docs/manpage.html>